

Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра ИИТ

Лабораторная работа №2

По дисциплине: «АПО ЭВМиС»

Тема: «Изучение аппаратно-программной архитектуры процессоров
семейства Intel - обработка символьных данных»

Выполнил:
Студент 4-го курса
Группы АС-66
Ярома А. И.
Проверил:
Кулик А. Д.

Брест 2026

Цель работы

Изучить программную модель процессора Intel и закрепить работу с регистровой и оперативной памятью при обработке символьной последовательности на ассемблере.

Задание. Вариант 12

Дан непустой текст длиной не более 100 символов. Ввод завершается точкой, которая в текст не включается. Проверить, равно ли количество строчных русских букв количеству цифр. Если условие истинно, циклически сдвинуть текст на три позиции влево и напечатать результат. Иначе вывести сообщение об отсутствии преобразования.

Особенности реализации

Программа состоит из последовательных этапов: ввод, проверка допустимости текста, проверка условия, преобразование и печать. Проверка условия `CheckConditionAsm` и сдвиг `RotateLeftAsm` реализованы как процедуры MASM x86. Оболочка C++ отвечает за ввод и вывод в UTF-8.

В рабочем буфере один `DWORD` представляет один символ Unicode. Так русская буква считается одним символом и сдвиг не разрывает многобайтовую кодировку. Буфер имеет постоянную ёмкость 101 элемент. Глобальные переменные и дополнительный массив для преобразования не используются.

Проверка учитывает а-я (`0430h-044Fh`), ё (`0451h`) и цифры 0-9. Прописные русские и латинские буквы не увеличивают счётчик строчных русских букв. Равенство $0 = 0$ удовлетворяет условию.

Эффективное число сдвигов равно $3 \bmod n$. На каждом шаге первый символ сохраняется в `EAX`, команда `REP MOVSD` переносит оставшиеся элементы влево, затем сохранённый символ записывается в конец. `CLD` задаёт движение по возрастающим адресам. Дополнительная память $O(1)$; время преобразования $O(n)$.

Текст программы

Ассемблерные процедуры: text_processing.asm

```
1 ; APO EVMiS, lab 2, variant 12. MASM, x86, cdecl.
2 ; Each DWORD stores one Unicode character. No global variables.
3 .386
4 .model flat, C
5 .code
6 ; int CheckConditionAsm(text, length, letters, digits)
7 ; Counts Russian lowercase letters (including yo) and ASCII digits.
8 ; EAX = 1 when the counts are equal, otherwise EAX = 0.
9 CheckConditionAsm PROC USES esi ebx,
10     pText:PTR DWORD, charCount:DWORD, pLetters:PTR DWORD, pDigits:PTR DWORD
11     mov esi, pText
12     mov ecx, charCount
13     xor ebx, ebx
14     xor edx, edx
15     cld
16     test ecx, ecx
17     jz counts_ready
18 check_character:
19     lodsd
20     cmp eax, 0430h           ; Russian lowercase a
21     jnb check_yo
22     cmp eax, 044Fh           ; Russian lowercase ya
23     jbe count_letter
24 check_yo:
25     cmp eax, 0451h           ; Russian lowercase yo
26     je count_letter
27     cmp eax, '0'
28     jnb next_character
29     cmp eax, '9'
30     ja next_character
31     inc edx
32     jmp next_character
33 count_letter:
34     inc ebx
35 next_character:
36     dec ecx
37     jnz check_character
38 counts_ready:
39     mov eax, pLetters
40     mov [eax], ebx
41     mov eax, pDigits
42     mov [eax], edx
43     xor eax, eax
44     cmp ebx, edx
45     sete al
46     ret
47 CheckConditionAsm ENDP
48 ; void RotateLeftAsm(text, length)
49 ; Three left rotations; effective count is 3 mod length.
50 ; A character is kept in EAX, the remaining characters move with REP MOVSD.
51 ; No second text buffer and no pushes inside the rotation loop.
52 RotateLeftAsm PROC USES esi edi ebx,
53     pText:PTR DWORD, charCount:DWORD
54     cld
55     mov ebx, charCount
56     cmp ebx, 1
57     jbe rotation_done
58     mov eax, 3
59     xor edx, edx
60     div ebx
61     mov ebx, edx             ; number of single-position rotations
62     test ebx, ebx
63     jz rotation_done
64 rotate_once:
65     mov edi, pText
66     mov eax, [edi]           ; retain the first character
67     lea esi, [edi + 4]
68     mov ecx, charCount
```

```

74     dec ecx
75     rep movsd          ; move text[1..n-1] to text[0..n-2]
76     mov [edi], eax      ; append the retained character
77     dec ebx
78     jnz rotate_once
80 rotation_done:
81     ret
82 RotateLeftAsm ENDP
83 END

```

Ввод и вывод: main.cpp

```

1  #define WIN32_LEAN_AND_MEAN
2  #define NOMINMAX
3  #include <Windows.h>
4  #include <cstdint>
5  #include <cstdint>
6  #include <iostream>
7  #if !defined(_MSC_VER) || !defined(_M_IX86)
8  #error Build this project with MSVC for Win32.
9  #endif
10 extern "C" int __cdecl CheckConditionAsm(const std::uint32_t* text,
11     std::size_t length, std::size_t* letters, std::size_t* digits);
12 extern "C" void __cdecl RotateLeftAsm(std::uint32_t* text, std::size_t length);
13 enum class InputStatus { Ok, Empty, TooLong, NoPeriod, InvalidUtf8 };
14 // Read one Unicode character without allocating a string or a second buffer.
15 bool ReadCharacter(std::istream& input, std::uint32_t& value)
16 {
17     const int first = input.get();
18     if (first == std::char_traits<char>::eof())
19         return false;
20     const auto byte = static_cast<unsigned char>(first);
21     unsigned remaining = 0;
22     std::uint32_t minimum = 0;
23     if (byte < 0x80) {
24         value = byte;
25         return true;
26     }
27     if (byte >= 0xC2 && byte <= 0xDF) {
28         value = byte & 0x1F; remaining = 1; minimum = 0x80;
29     } else if (byte >= 0xE0 && byte <= 0xEF) {
30         value = byte & 0x0F; remaining = 2; minimum = 0x800;
31     } else if (byte >= 0xF0 && byte <= 0xF4) {
32         value = byte & 0x07; remaining = 3; minimum = 0x10000;
33     } else {
34         return false;
35     }
36     while (remaining-- != 0) {
37         const int next = input.get();
38         if (next == std::char_traits<char>::eof() || (next & 0xC0) != 0x80)
39             return false;
40         value = (value << 6) | (next & 0x3F);
41     }
42     return value >= minimum && value <= 0x10FFFF
43         && !(value >= 0xD800 && value <= 0xDFFF);
44 }
45 InputStatus ReadText(std::istream& input, std::uint32_t* text,
46     std::size_t capacity, std::size_t& length)
47 {
48     length = 0;
49     bool overflow = false;
50     for (;;) {
51         if (input.peek() == std::char_traits<char>::eof())
52             return InputStatus::NoPeriod;
53         std::uint32_t character = 0;
54         if (!ReadCharacter(input, character))
55             return InputStatus::InvalidUtf8;
56         if (character == '.') {
57             text[length] = 0;
58             if (overflow)
59                 return InputStatus::TooLong;
60             return length == 0 ? InputStatus::Empty : InputStatus::Ok;
61         }
62     }
63 }

```

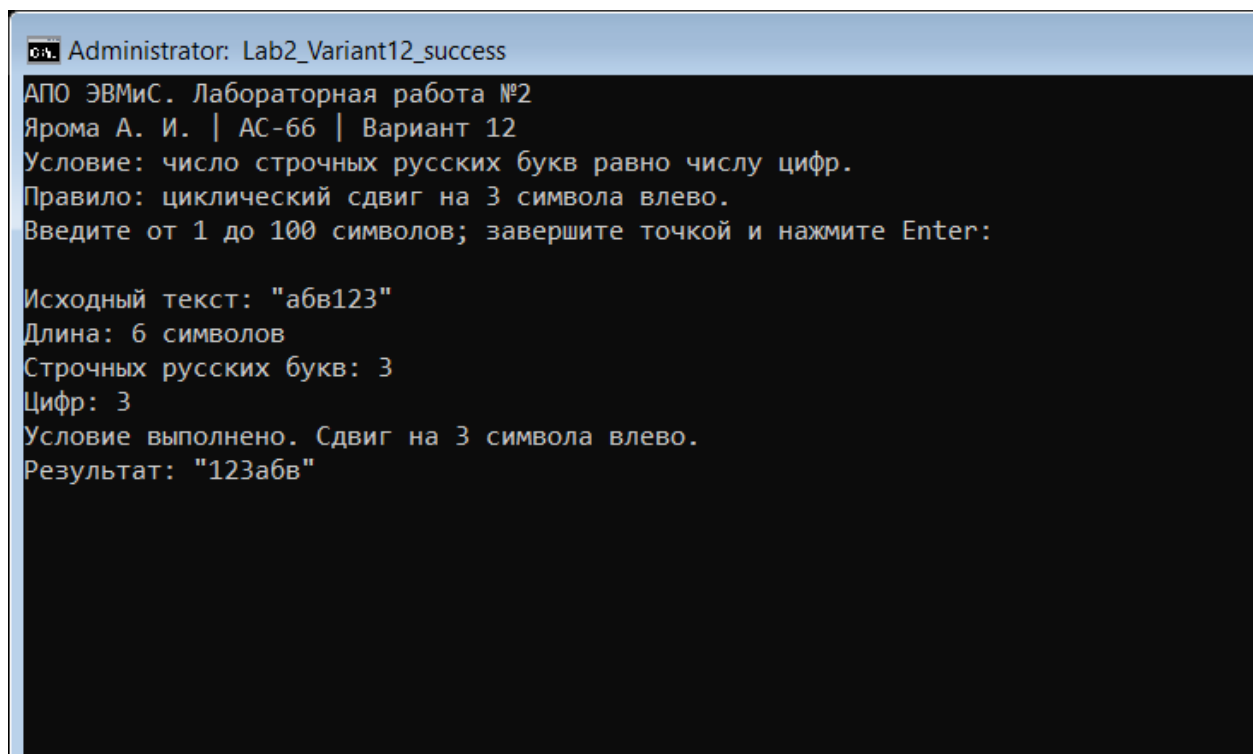
```

68     if (length < capacity)
69         text[length++] = character;
70     else
71         overflow = true;
72 }
73 }
74
75 void PrintText(const std::uint32_t* text, std::size_t length)
76 {
77     for (std::size_t index = 0; index < length; ++index) {
78         const std::uint32_t value = text[index];
79         auto byte = [](std::uint32_t part) {
80             std::cout.put(static_cast<char>(part));
81         };
82         if (value < 0x80) {
83             byte(value);
84         } else if (value < 0x800) {
85             byte(0xC0 | (value >> 6)); byte(0x80 | (value & 0x3F));
86         } else if (value < 0x10000) {
87             byte(0xE0 | (value >> 12));
88             byte(0x80 | ((value >> 6) & 0x3F)); byte(0x80 | (value & 0x3F));
89         } else {
90             byte(0xF0 | (value >> 18)); byte(0x80 | ((value >> 12) & 0x3F));
91             byte(0x80 | ((value >> 6) & 0x3F)); byte(0x80 | (value & 0x3F));
92         }
93     }
94 }
95
96 int main()
97 {
98     SetConsoleCP(CP_UTF8);
99     SetConsoleOutputCP(CP_UTF8);
100     constexpr std::size_t MaxLength = 100;
101     std::uint32_t text[MaxLength + 1] = {};
102     std::size_t length = 0;
103     std::cout << "АПО ЭВМиС. Лабораторная работа №2\n"
104     "Ярома А. И. | АС-66 | Вариант 12\n"
105     "Условие: число строчных русских букв равно числу цифр.\n"
106     "Правило: циклический сдвиг на 3 символа влево.\n"
107     "Введите от 1 до 100 символов; завершите точкой и нажмите Enter:\n";
108     const InputStatus status = ReadText(std::cin, text, MaxLength, length);
109     if (status != InputStatus::Ok) {
110         std::cout << "Последовательность не является допустимым текстом: ";
111         switch (status) {
112             case InputStatus::Empty: std::cout << "пустой текст.\n"; break;
113             case InputStatus::TooLong: std::cout << "больше 100 символов.\n"; break;
114             case InputStatus::NoPeriod: std::cout << "нет завершающей точки.\n"; break;
115             default: std::cout << "ошибка кодировки UTF-8.\n"; break;
116         }
117         std::cout << "Преобразование не выполняется.\n";
118         return 1;
119     }
120
121     std::cout << "\nИсходный текст: \n";
122     PrintText(text, length);
123     std::cout << "\nДлина: " << length << " символов\n";
124     std::size_t letters = 0, digits = 0;
125     const bool satisfied = CheckConditionAsm(text, length, &letters, &digits) != 0;
126     std::cout << "Строчных русских букв: " << letters
127     << "\nЦифр: " << digits << '\n';
128     if (!satisfied) {
129         std::cout << "Условие не выполнено. Текст не преобразуется.\n";
130         return 0;
131     }
132
133     std::cout << "Условие выполнено. Сдвиг на 3 символа влево.\n";
134     RotateLeftAsm(text, length);
135     std::cout << "Результат: \n";
136     PrintText(text, length);
137     std::cout << "\n\n";
138     return 0;
139 }

```

Результаты выполнения

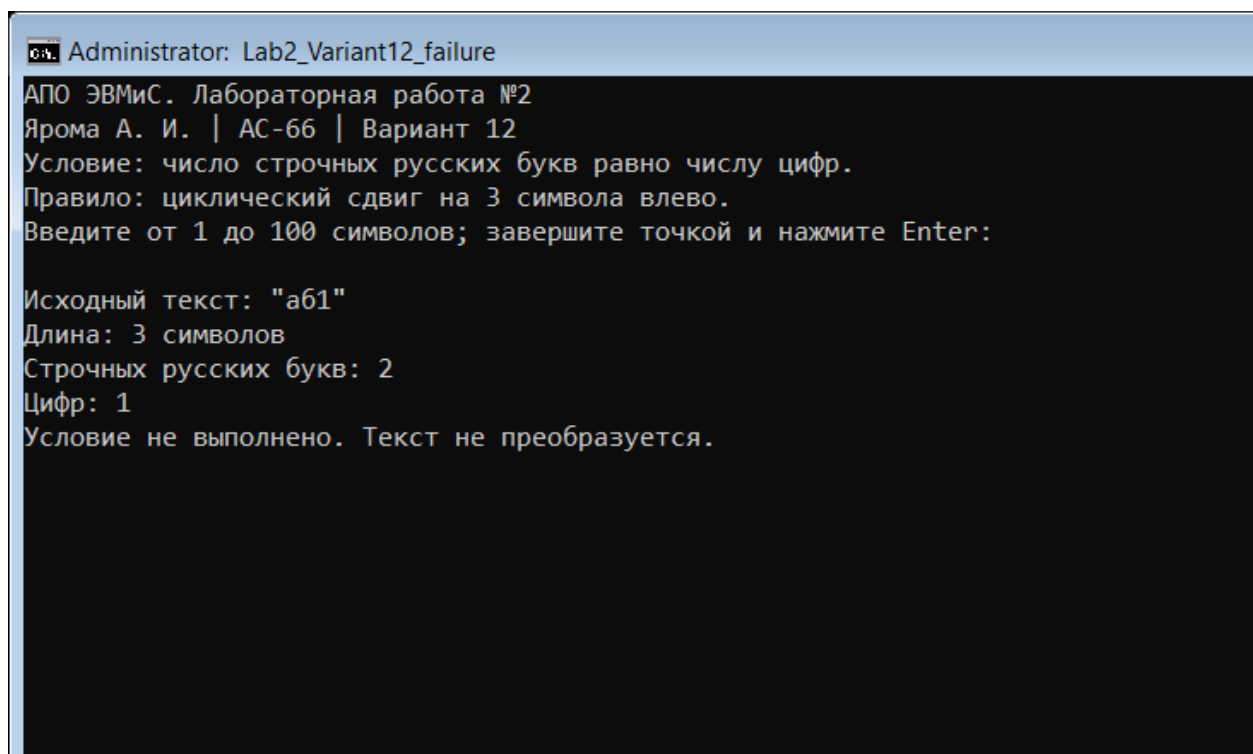
Скриншоты получены при запуске собранной программы в конфигурации Release / Win32. Ввод передан из файлов UTF-8; исходная последовательность видна в выводе программы.



```
Administrator: Lab2_Variant12_success
АПО ЭВМиС. Лабораторная работа №2
Ярома А. И. | АС-66 | Вариант 12
Условие: число строчных русских букв равно числу цифр.
Правило: циклический сдвиг на 3 символа влево.
Введите от 1 до 100 символов; завершите точкой и нажмите Enter:

Исходный текст: "абв123"
Длина: 6 символов
Строчных русских букв: 3
Цифр: 3
Условие выполнено. Сдвиг на 3 символа влево.
Результат: "123абв"
```

Рисунок 1 - «абв123.»: три буквы и три цифры; результат «123абв».



```
Administrator: Lab2_Variant12_failure
АПО ЭВМиС. Лабораторная работа №2
Ярома А. И. | АС-66 | Вариант 12
Условие: число строчных русских букв равно числу цифр.
Правило: циклический сдвиг на 3 символа влево.
Введите от 1 до 100 символов; завершите точкой и нажмите Enter:

Исходный текст: "аб1"
Длина: 3 символов
Строчных русских букв: 2
Цифр: 1
Условие не выполнено. Текст не преобразуется.
```

Рисунок 2 - «аб1.»: две буквы и одна цифра; преобразования нет.

Граничные случаи и проверка

```
Administrator: Lab2_Variant12_short
АПО ЭВМиС. Лабораторная работа №2
Ярома А. И. | АС-66 | Вариант 12
Условие: число строчных русских букв равно числу цифр.
Правило: циклический сдвиг на 3 символа влево.
Введите от 1 до 100 символов; завершите точкой и нажмите Enter:

Исходный текст: "ё1"
Длина: 2 символов
Строчных русских букв: 1
Цифр: 1
Условие выполнено. Сдвиг на 3 символа влево.
Результат: "1ё"
```

Рисунок 3 - «ё1.»: $3 \bmod 2 = 1$; результат «1ё». Буква ё учтена.

Ввод	Ожидаемое поведение
ABC.	0 букв = 0 цифр; длина 3, результат ABC
AB.	Сдвиг на одну позицию; результат BA
.	Пустой текст; преобразование не выполняется
101 символ и точка	Превышение лимита; преобразования нет
abc без точки	Нет завершающей точки; преобразования нет
100 символов и точка	Допустимая максимальная длина

В Debug и Release прошли по 185 сценариев ввода: 23 заданных текста, 150 случайных, 11 ошибочных последовательностей и проверка окончания на первой точке. Дополнительно выполнены 1010 прямых тестов процедур для длин от 0 до 100. Проверены оба счётчика, неизменность буфера после проверки условия, результат сдвига и сохранность контрольных элементов за границами текста.

Вывод

Разработана программа варианта 12: ассемблерная процедура сравнивает число строчных русских букв и цифр, а при истинном условии другая процедура циклически сдвигает текст влево на три позиции. Использована строковая команда REP MOVSD. Проверки подтвердили корректную обработку русских букв, коротких строк и предельной длины, а также отказ от преобразования при недопустимом вводе.